Practical 2 – AI – I Miss My Cats

250038453

Overview

This project was built on a previous game that implemented a basic physics engine and collision system with an overall goal to add characters and basic character AI to the game. This AI was added through a new NPC represented as cat ghosts that chase the player character around the screen. These ghosts use the tile map to find paths through passable tiles to get to the player character. Additionally, the ghosts move on the Tile Map and are drawn using the offset gotten from the player character's movement. They also use the Tile Map to determine collisions with the environment and the player.

Design

The cat ghosts in Cat Game use various mechanics to fully work within the game including steering behaviors, behavior states, and pathing. In terms of steering behaviors ghosts can either chase or flee from the player and use a separation behavior to stay a distance away from each other. This causes the ghosts to follow unique paths when it makes sense and gives the player a more challenging experience when trying to complete the level. Also, ghosts have various behavior states including IDLE, DEAD, CHASE, FLEE. These states determine how ghosts behave on screen and how they interact with the character. An IDLE and DEAD mostly determine how the game handles the ghost and CHASE and FEE mostly determine how the ghosts behave. Finally, the state CHASE activates the chase function which uses pathfinding to route the ghost to the player character. The path-finding algorithm uses the Tile Map to determine the fastest path the ghost needs to take to reach the player character. It can create a

path using both adjacent and diagonal tiles, and chase() uses this path to determine the ghost's location.

Technical Description

There were three main mechanics added to Cat Game that influence character AI, steering behaviors, behavior states, and A* pathfinding. First, the ghosts move in a few different ways chasing, fleeing and separating from each other. The default behavior of ghosts is to chases the player when the ghosts first spawn on the screen. The path that the ghosts use to chase the player is determined by an A* algorithm that is performed inside the Tile Map class. Of course, there is also a basic backup behavior that is used if a valid path cannot be found. Like the flee behavior, this backup chase behavior is a steering mechanism that adjusts the ghost's velocity to naturally follow the player. I implemented chase and flee as steering behavior over kinematic behavior to make the movement of the ghosts look more natural and realistic. Additionally, every frame a function called seperateBehavior is run which adjusts the velocity of the ghosts so that they fly separately from one another rather than in the same path. This adds to the realism of the ghost movement by forcing different ghosts to potentially take different paths.

To track and control how the world interacts with a character and how the character interacts with the world I used the behavior states IDLE, DEAD, CHASE, and FLEE. IDLE and DEAD are conditions that tell the game how to handle the ghosts. IDLE simple means that the ghost is inactive, and tell the game that it does not have to run any pathing or physics operations on the character. DEAD is a state that occurs when the character is no longer on the screen for 5 seconds or when the game restarts. This simply tells the game that the character is no longer active and no pathing or movement needs to happen on them. They are also no longer drawn. Once a character is on the screen for the first time the movement behaviors CHASE and FLEE

are activated. CHASE is the behavior that is automatically activated and simply sets the ghost to chase the cat around. FLEE essentially does the opposite of CHASE. Both these behaviors can switch change to the next state, FLEE to CHASE or CHASE to FLEE. This is dependent on the sleeping status of the cat player character, Poe. When Poe switches to his invincible sleep state, the ghosts flee because they no longer have a reason to attack him and he is a natural anomaly. When this state is off (he's running normally) the ghosts switch back to chasing him.

The chase system implements an A* pathing algorithm to determine the best path for the ghosts to follow to reach the player. The main premise of this algorithm is to check tiles, tile by tile, by how close they are to the goal position and calculate the fastest path to the goal. This path is mostly made of adjacent tiles but can also include diagonal tiles if there are no impassible adjacent tiles to the diagonal tile. Additionally, this algorithm checks if another entity is in its path and chooses another path that the other entity is not in. This helps combine separation behavior with the pathing behavior without a ton of ghosts getting stuck against each other. The path that is found is then passed as a stack to the chase function which uses each point of the path to adjust the ghost's velocity. This path is also reset every 30 frames to keep up with the player's movement. This pathing algorithm was the most complicated to implement cleanly because it has many facets that dictate the ghost's movement. Throughout testing the ghosts often got stuck on walls or on each other when an opening was too small so many adjustments had to be made so that the ghosts would function properly.

Discussion and Conclusion

Similar to the last practical, this practical brought its own collection of various challenges. I found it very difficult to get the ghost movement looking natural because of

the frequently changing paths, and the fluctuation in the velocity. With ghost movement, it was also hard to get the ghosts to move in their own paths without getting stuck on one another. If two ghosts were trying to enter a tile sized hole together, they would not be able to because of the separation behavior. Additionally, it was difficult to keep track of how to translate the tile map with the physics of how the ghosts were moving and where the path was. In the future, I would like to improve the movement of the ghosts, so that they move more elegantly through the level, especially when it is purposely made to trick them. I also wish I was able to add more characters to the game. I wanted to add a water cannon that shot water bullets and a vacuum cleaner, but I didn't end up having enough time to add these extra elements. In this work, I focused on the ghosts because I was able to logically implement each level of the practical with them. The vacuum and the cannon would be less complex.

Overall, I feel good about how I implemented AI into my game with the cat ghosts and they make the game significantly more fun. Plus, now Poe's ability has a purpose. While there are many improvements I still want to make, I feel like my implementation accurately shows my knowledge of how AI works in a video game. Also, I have never implemented an A* algorithm before, so it was very informative to learn how to do a new and useful algorithm. I also enjoyed implementing this algorithm through the logical run down I found on Geeks for Geeks, it really helped me understand the algorithm and how it could work for my use case.

References

A* Concept *https://www.geeksforgeeks.org/dsa/a-search-algorithm/*

CS4303 Lecture Materials and Examples https://studres.cs.st-andrews.ac.uk/CS4303/Lectures/